ACID Properties

PostgreSQL vs ClickHouse

A cleaned and polished summary of how each database approaches transactional reliability and performance trade-offs.

Executive summary

ACID stands for Atomicity, Consistency, Isolation, and Durability. PostgreSQL is built for transactional correctness and strict database guarantees. ClickHouse is built for analytical speed, so it accepts different trade-offs: fewer traditional transaction features, but much faster large-scale analytical reads and aggregations.

PostgreSQL	OLTP database focused on transactional correctness and ACID behavior.
ClickHouse	OLAP database focused on high-speed analytical queries over large datasets.
Main trade-off	PostgreSQL prioritizes correctness across transactions; ClickHouse prioritizes analytical performance and throughput.

What is ACID?

ACID is a set of properties that helps ensure database transactions are processed reliably. A transaction is a logical operation, or a group of operations, that must be treated as one unit.

Atomicity	The transaction either succeeds completely or fails completely. There should be no partial save.
Consistency	A transaction moves the database from one valid state to another while preserving predefined rules.
Isolation	Concurrent transactions should not expose intermediate or uncommitted states to one another.
Durability	Once a transaction is committed, it should survive failures such as crashes or power loss.

PostgreSQL: transactional correctness first

PostgreSQL is designed mainly for Online Transactional Processing (OLTP). In this type of workload, correctness, constraints, and reliable transaction behavior are usually more important than raw analytical scan speed.

- Atomicity is supported through transactions and Write-Ahead Logging (WAL). Changes are logged before they are applied, which allows rollback when an error occurs.
- Consistency is enforced by the engine using primary keys, foreign keys, check constraints, and other database rules.
- Isolation is implemented using Multi-Version Concurrency Control (MVCC), so readers and writers can work concurrently without exposing uncommitted intermediate states.
- Durability is supported through WAL and crash recovery. After a committed transaction, PostgreSQL can replay logs to restore a consistent state after failure.

ClickHouse: analytical speed first

ClickHouse is a column-oriented database designed mainly for Online Analytical Processing (OLAP). It is optimized for scanning and aggregating very large datasets quickly. That design gives excellent analytical performance, but it is not meant to behave like a traditional OLTP transaction engine.

- Atomicity is mainly provided for insert blocks: a batch insert is written as a single part. However, ClickHouse does not support general multi-statement transactions across multiple tables.
- Consistency is mainly structural: schemas are enforced, but relational constraints such as foreign keys are not the central mechanism. Data validation is often handled by the application or ETL layer.
- Isolation is different from PostgreSQL: ClickHouse stores immutable parts and applies updates through background mutations. It is not designed around standard SQL transaction isolation levels or MVCC.
- Durability is based on storing data as persistent MergeTree parts, often with replication in production setups. The system is optimized for batch ingestion, not many tiny OLTP-style writes.

Side-by-side comparison

Property	PostgreSQL	ClickHouse
Main workload	OLTP: inserts, updates, constraints, and transactional applications.	OLAP: large scans, aggregations, reporting, and analytical queries.
Atomicity	Transactions are handled as complete units. WAL allows rollback if an operation fails.	Single insert blocks are atomic, but ClickHouse does not provide general multi-statement transactions like BEGIN/COMMIT across tables.
Consistency	Primary keys, foreign keys, check constraints, and other rules are enforced by the database engine.	Schemas are enforced, but relational constraints are limited. More validation is usually pushed to the application or ETL pipeline.
Isolation	MVCC supports concurrent reads and writes while protecting users from intermediate uncommitted states.	Data is stored in immutable parts and updates are usually background mutations. It is not designed around full SQL transaction isolation levels.
Durability	Committed changes are protected by WAL and crash recovery mechanisms.	MergeTree stores data as persistent parts and can use replication, but the write path is optimized for batch ingestion rather than tiny OLTP-style writes.
Best fit	Banking, orders, inventory, user records, and any workload where transaction correctness is central.	Logs, events, telemetry, dashboards, reporting, and benchmark-style analytical queries.

Bottom line

The important conclusion

PostgreSQL and ClickHouse are not trying to solve the same problem. PostgreSQL is the stronger choice when transaction correctness is the priority. ClickHouse is the stronger choice when the workload is analytical, read-heavy, and performance over large datasets matters most.

Connection to the benchmark project

This architectural difference explains why ClickHouse can outperform PostgreSQL in analytical benchmark queries: it uses a columnar storage model and performance-focused execution path, while PostgreSQL keeps stronger transactional machinery for OLTP correctness.